

F450 Drone PID Cascade Controller

Tuan Anh - dolphiinn

July 2, 2026

Contents

Chapter 1: Cơ sở lý thuyết và thiết kế controller	3
1.1 Kiến trúc tổng thể của bộ điều khiển F450	3
1.1.1 Luồng dữ liệu điều khiển	3
1.1.2 Vì sao dùng cấu trúc cascade	4
1.1.3 Chu kỳ điều khiển trong <code>on_physics_step</code>	4
1.1.4 Các tham số mặc định quan trọng	5
1.1.5 Các hàm tiện ích chung	5
1.2 Position Hold PID	6
1.2.1 Công thức PID chuẩn	6
1.2.2 Sai số vị trí trong Position Hold	6
1.2.3 Sai số vận tốc	7
1.2.4 Tích phân sai số	7
1.2.5 PID vị trí tạo gia tốc mong muốn	8
1.2.6 Bản chất vật lý của việc chọn gia tốc làm output	8
1.2.7 Giới hạn gia tốc ngang	9
1.2.8 Đổi từ [W] sang [B]	9
1.2.9 Bản chất vật lý - nghiêng drone để tạo gia tốc ngang	10
1.2.10 Tính pitch target	11
1.2.11 Tính roll target	11
1.2.12 Giới hạn góc nghiêng	12
1.2.13 Công thức tổng hợp của Position Hold PID	12
1.3 Altitude Hold PID	14
1.3.1 Sai số độ cao và vận tốc đứng	14
1.3.2 Tích phân độ cao	14
1.3.3 Output của Altitude PID	14
1.3.4 Ý nghĩa vật lý	15
1.4 Attitude Hold PID	15
1.4.1 Sai số roll/pitch	15
1.4.2 Thành phần vi phân dùng vận tốc góc	15
1.4.3 Tích phân và giới hạn output	16
1.5 Yaw Hold PID	16
1.5.1 Sai số yaw	16
1.5.2 Công thức yaw correction	17
1.5.3 Vai trò của yaw correction trong quadcopter	17
1.6 Quad X PWM Mixer	18
1.6.1 Công thức mixer	18
1.6.2 Ý nghĩa dấu trong mixer	18

	1.6.2.1	Quy ước hệ trục và vị trí motor	19
	1.6.2.2	Vì sao chênh lệch lực tạo ra moment?	19
	1.6.2.3	Ý nghĩa của PWM nền	20
	1.6.2.4	Ý nghĩa chung của các delta PWM	20
	1.6.2.5	Roll correction tạo moment roll	21
	1.6.2.6	Vì sao roll correction không làm đổi nhiều tổng lực nâng?	22
	1.6.2.7	Pitch correction tạo moment pitch	22
	1.6.2.8	Yaw correction tạo moment yaw	23
1.7	Motor Model		26
	1.7.1	Đổi PWM sang throttle	26
	1.7.2	Dòng điện và torque motor	27
	1.7.3	Động lực học rotor	27
	1.7.4	Lực đẩy, torque phản lực và RPM	28
1.8	Apply lực vào Isaac Sim		28
	1.8.1	Torque phản lực yaw	28
	1.8.2	Kết nối giữa motor model và simulator	29
1.9	Cách xác định output của một bộ PID		30
	1.9.1	Output của các tầng PID trong project	30
	1.9.1.1	Position PID	30
	1.9.1.2	Altitude PID	30
	1.9.1.3	Attitude PID	31
	1.9.1.4	Yaw PID	31
	1.9.2	Mixer	32
	1.9.3	Tóm tắt nguyên tắc chọn output PID	32
1.10	Propeller Spinner		34
	1.10.1	Đổi RPM sang vận tốc joint	34
1.11	Tracking Logger		34
1.12	Compatibility Mixin		35
1.13	Tóm tắt toàn bộ hệ thống		35
Chapter 2: Sử dụng và chạy mô phỏng trong Isaac Sim			37
2.1	Tạo cây thư mục giống project		37
	2.1.1	Cấu trúc thư mục chính	37
	2.1.2	Tạo workspace và package	38
2.2	Import file URDF vào Isaac Sim		39
2.3	Viết controller ngoài và gọi từ Script Editor		39
	2.3.1	Setup path và import controller	40
	2.3.2	Reload module khi sửa code	40
	2.3.3	Tạo controller và bắt đầu chạy	41
2.4	Mở, chạy mô phỏng, log data và vẽ đồ thị		41
	2.4.1	Mở mô phỏng	41
	2.4.2	Đặt setpoint tĩnh	42
	2.4.3	Bay theo quỹ đạo tròn	42
	2.4.4	Bật và dừng log	43
	2.4.5	Vẽ đồ thị realtime	43
	2.4.6	Vẽ đồ thị 2D sau mô phỏng	43
	2.4.7	Vẽ quỹ đạo 3D	44
	2.4.8	Dừng controller	44

Chapter 1: Cơ sở lý thuyết và thiết kế controller

1.1 Kiến trúc tổng thể của bộ điều khiển F450

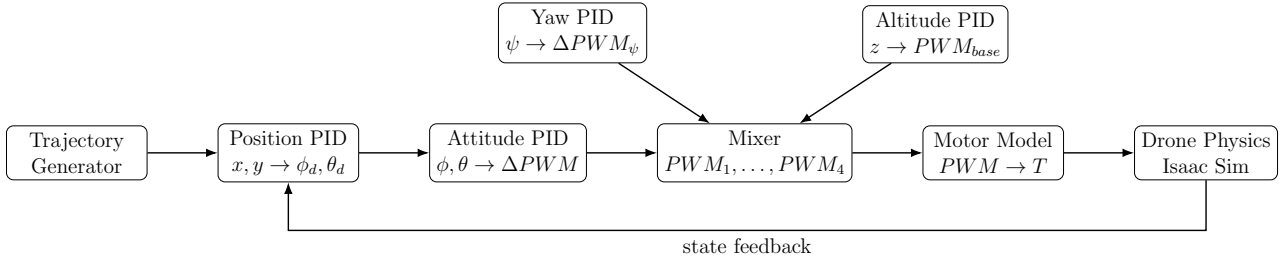


Figure 1: Cascade PID control structure

Phần điều khiển chính của drone F450 được tổ chức theo dạng *cascade PID controller*. Lớp trung tâm là `F450AttitudeHold` trong file `issac_attitude_hold.py`. Lớp này không chứa toàn bộ thuật toán trong một khối lớn, mà ghép nhiều module nhỏ lại với nhau:

- `PositionHoldPID`: giữ vị trí ngang x, y và tạo $roll_target, pitch_target$.
- `AltitudeHoldPID`: giữ độ cao z và tạo PWM_{base} .
- `AttitudeHoldPID`: giữ roll/pitch và tạo $\Delta PWM_{\phi}, \Delta PWM_{\theta}$.
- `YawHoldPID`: giữ yaw và tạo ΔPWM_{ψ} .
- `QuadXPwmMixer`: trộn PWM base với các PWM hiệu chỉnh - ΔPWM để tạo bốn lệnh motor.
- `MotorModel`: đổi PWM thành lực đẩy, dòng điện, RPM và torque phản lực của từng motor.
- `PhysicalPropellerSpinner`: làm quay joint cánh quạt vật lý theo RPM mô phỏng.
- `TrackingLogger`: ghi trạng thái và target ra file CSV để phân tích sau khi bay.

1.1.1 Luồng dữ liệu điều khiển

Trong mỗi bước vật lý của Isaac Sim, controller đọc trạng thái hiện tại của drone:

$$x, y, z, v_x, v_y, v_z, \phi, \theta, \psi, p, q, r \quad (1)$$

Trong đó ϕ là roll, θ là pitch, ψ là yaw, còn p, q, r là vận tốc góc quanh các trục body. Sau đó luồng điều khiển chính là:

$$x_d, y_d, x, y, v_x, v_y, \psi \rightarrow PositionPID \rightarrow \phi_d, \theta_d \quad (2)$$

$$z_d, z, v_z \rightarrow AltitudePID \rightarrow PWM_{base} \quad (3)$$

$$\phi_d, \theta_d, \phi, \theta, p, q \rightarrow AttitudePID \rightarrow \Delta PWM_{\phi}, \Delta PWM_{\theta} \quad (4)$$

$$\psi_d, \psi, r \rightarrow YawPID \rightarrow \Delta PWM_{\psi} \quad (5)$$

Các output trên được đưa vào mixer:

$$PWM_{base}, \Delta PWM_{\phi}, \Delta PWM_{\theta}, \Delta PWM_{\psi} \longrightarrow Mixer \longrightarrow PWM_1, PWM_2, PWM_3, PWM_4 \quad (6)$$

Cuối cùng, từng lệnh PWM đi qua motor model để tạo lực đẩy:

$$PWM_i \longrightarrow MotorModel_i \longrightarrow T_i, RPM_i, Q_i \quad (7)$$

Trong đó T_i là lực đẩy của motor thứ i , còn Q_i là torque phản lực do cánh quạt tạo ra.

1.1.2 Vì sao dùng cấu trúc cascade

Drone là hệ động lực học nhiều tầng. Vị trí không thể điều khiển trực tiếp bằng PWM, vì PWM trước hết tạo lực đẩy, lực đẩy tạo moment và gia tốc, rồi gia tốc mới làm thay đổi vị trí. Vì vậy project chia bài toán thành nhiều tầng:

- Tầng ngoài cùng - outer loop quyết định drone cần nghiêng bao nhiêu để quay về vị trí mong muốn.
- Tầng độ cao quyết định tổng lực nâng cần tăng hay giảm bao nhiêu.
- Tầng attitude làm drone bám theo góc roll/pitch mong muốn.
- Tầng yaw giữ hướng quay quanh trục đứng.
- Mixer biến các lệnh trừu tượng thành lệnh cụ thể cho bốn motor.

Cách chia tầng này giúp mỗi PID có một nhiệm vụ rõ ràng. Position PID không cần biết chi tiết từng motor; attitude PID không cần biết target vị trí đến từ đâu; motor model chỉ cần nhận PWM và trả về lực.

1.1.3 Chu kỳ điều khiển trong on_physics_step

Hàm `on_physics_step(dt)` là vòng lặp điều khiển được gọi mỗi bước physics. Các bước chính là:

1. Lấy rigid body `base_link` từ `dynamic_control`.
2. Đọc pose, vận tốc tuyến tính và vận tốc góc.
3. Đổi quaternion sang Euler bằng `quat_to_euler`.
4. Nếu chưa đặt target x, y , tự lấy vị trí ban đầu làm target giữ vị trí.
5. Tính target roll/pitch từ position controller.
6. Tính PWM_{base} từ altitude controller.
7. Tính yaw correction nếu yaw hold đang bật.
8. Tính roll/pitch correction từ attitude controller.
9. Trộn bốn lệnh PWM bằng mixer.
10. Cập nhật bốn motor model và apply lực đẩy vào Isaac Sim.
11. Tính torque phản lực yaw, quay cánh quạt vật lý, ghi log và in debug.

Như vậy, Isaac Sim vừa là môi trường vật lý nhận lực từ controller, vừa là cảm biến trả lại trạng thái cho controller ở bước sau.

1.1.4 Các tham số mặc định quan trọng

Nhóm	Tham số	Giá trị mặc định trong code
PWM	PWM_{min}, PWM_{max}	1100, 1940
F450 main	z_d	1.5 m
F450 main	PWM_{hover}	1650
Position	$K_{p_x}, K_{d_x}, K_{i_x}$	0.55, 1.30, 0.03
Position	$K_{p_y}, K_{d_y}, K_{i_y}$	0.55, 1.30, 0.03
Position	giới hạn gia tốc	1.20 m/s ²
Position	giới hạn góc	6.5°
Altitude	$K_{p_z}, K_{d_z}, K_{i_z}$	220, 150, 15
Attitude	K_p, K_i, K_d roll/pitch	520, 25, 130
Yaw	$K_{p_\psi}, K_{i_\psi}, K_{d_\psi}$	95, 8, 28

Lưu ý: các giá trị này có thể thay đổi trong lúc chạy bằng cách áp đặt giá trị mới vào Script Editor của Isaac Sim.

1.1.5 Các hàm tiện ích chung

File `control_utils.py` chứa các hàm nhỏ nhưng quan trọng:

$$\text{clamp}(x, l, h) = \max(l, \min(x, h)) \quad (8)$$

Hàm này dùng để giới hạn PWM, góc, gia tốc và tích phân PID trong vùng an toàn.

Hàm `wrap_angle` đưa góc về khoảng:

$$-\pi \leq \text{angle} \leq \pi \quad (9)$$

Nhờ đó sai số góc luôn lấy theo đường ngắn nhất. Ví dụ, sai số giữa 179° và −179° phải là 2°, không phải 358°.

Hàm `quat_to_euler` đổi quaternion của Isaac Sim sang roll, pitch, yaw. Đây là bước cần thiết vì controller attitude và yaw làm việc trực tiếp với góc Euler.

1.2 Position Hold PID



Figure 2: Position PID control law

1.2.1 Công thức PID chuẩn

Công thức PID tổng quát có dạng [1]:

$$u(t) = K_p e(t) + K_i \int e(t) dt + K_d \dot{e}(t) \quad (10)$$

trong đó:

$$e(t) = r(t) - y(t) \quad (11)$$

Với:

- $r(t)$: giá trị mong muốn,
- $y(t)$: giá trị đo được,
- $e(t)$: sai số,
- $u(t)$: output điều khiển,
- K_p, K_i, K_d : hệ số tỉ lệ, tích phân và vi phân.

Trong bộ điều khiển drone, output $u(t)$ không nhất thiết là PWM. Tùy tầng điều khiển, $u(t)$ có thể là gia tốc, góc mong muốn, moment, lực nâng hoặc PWM.

1.2.2 Sai số vị trí trong Position Hold

Gọi vị trí mong muốn của drone trong WORLD-coordination [W] là:

$$x_d, \quad y_d \quad (12)$$

Vị trí hiện tại của drone là:

$$x, \quad y \quad (13)$$

Sai số vị trí được định nghĩa là:

$$e_x = x_d - x \quad (14)$$

$$e_y = y_d - y \quad (15)$$

Nếu $e_x > 0$, drone đang nằm phía sau target theo trục x và cần tạo gia tốc theo chiều $+x$. Nếu $e_y > 0$, drone đang lệch so với target theo trục y và cần tạo gia tốc theo chiều $+y$.

1.2.3 Sai số vận tốc

Gọi vận tốc mong muốn là:

$$v_{xd}, \quad v_{yd} \quad (16)$$

Vận tốc hiện tại là:

$$v_x, \quad v_y \quad (17)$$

Sai số vận tốc là:

$$e_{v_x} = v_{xd} - v_x \quad (18)$$

$$e_{v_y} = v_{yd} - v_y \quad (19)$$

Ta có:

$$e_x = x_d - x \quad (20)$$

Lấy đạo hàm hai vế theo thời gian:

$$\dot{e}_x = \dot{x}_d - \dot{x} \quad (21)$$

Mà:

$$\dot{x}_d = v_{xd} \quad (22)$$

$$\dot{x} = v_x \quad (23)$$

Do đó:

$$\dot{e}_x = v_{xd} - v_x = e_{v_x} \quad (24)$$

Tương tự:

$$\dot{e}_y = v_{yd} - v_y = e_{v_y} \quad (25)$$

Vì vậy, thành phần sai số vận tốc e_{v_x} và e_{v_y} chính là thành phần vi phân của sai số vị trí.

1.2.4 Tích phân sai số

Thành phần tích phân của sai số vị trí là:

$$I_x = \int e_x dt \quad (26)$$

$$I_y = \int e_y dt \quad (27)$$

Trong dạng rời rạc:

$$I_x(k) = I_x(k-1) + e_x(k)\Delta t \quad (28)$$

$$I_y(k) = I_y(k-1) + e_y(k)\Delta t \quad (29)$$

Để tránh hiện tượng tích phân tăng quá lớn, ta giới hạn:

$$-I_{\max} \leq I_x \leq I_{\max} \quad (30)$$

$$-I_{\max} \leq I_y \leq I_{\max} \quad (31)$$

Giới hạn này giúp chống hiện tượng *integral windup*.

1.2.5 PID vị trí tạo gia tốc mong muốn

Trong tầng Position Hold, PID không tạo PWM trực tiếp. Output của PID vị trí là gia tốc ngang mong muốn:

$$a_{x,cmd} \quad (32)$$

$$a_{y,cmd} \quad (33)$$

Công thức PID theo trục x :

$$a_{x,cmd} = K_{p_x} e_x + K_{d_x} \dot{e}_x + K_{i_x} I_x \quad (34)$$

Công thức PID theo trục y :

$$a_{y,cmd} = K_{p_y} e_y + K_{d_y} \dot{e}_y + K_{i_y} I_y \quad (35)$$

So với công thức PID chuẩn:

$$u = K_p e + K_i \int e dt + K_d \dot{e} \quad (36)$$

thì trong tầng Position Hold:

$$u_x = a_{x,cmd} \quad (37)$$

$$u_y = a_{y,cmd} \quad (38)$$

Tức là output của PID vị trí là gia tốc ngang mong muốn, không phải PWM.

1.2.6 Bản chất vật lý của việc chọn gia tốc làm output

Theo định luật II Newton:

$$F = ma \quad (39)$$

Muốn thay đổi vị trí ngang của drone, ta cần tạo gia tốc ngang:

$$a_x, \quad a_y \quad (40)$$

Do đó, tầng Position Hold trả lời câu hỏi:

Drone cần gia tốc ngang bao nhiêu để quay về vị trí mong muốn?

Vì vậy, output hợp lý của tầng Position Hold là:

$$a_{x,cmd}, \quad a_{y,cmd} \quad (41)$$

Sau đó, các gia tốc này sẽ được chuyển thành góc nghiêng mong muốn.

1.2.7 Giới hạn gia tốc ngang

Vector gia tốc ngang mong muốn là:

$$\mathbf{a}_{cmd} = \begin{bmatrix} a_{x,cmd} \\ a_{y,cmd} \end{bmatrix} \quad (42)$$

Độ lớn vector gia tốc là:

$$\|\mathbf{a}_{cmd}\| = \sqrt{a_{x,cmd}^2 + a_{y,cmd}^2} \quad (43)$$

Project giới hạn gia tốc ngang theo điều kiện:

$$\|\mathbf{a}_{cmd}\| \leq a_{\max} \quad (44)$$

Nếu:

$$\sqrt{a_{x,cmd}^2 + a_{y,cmd}^2} > a_{\max} \quad (45)$$

thì scale lại:

$$a_{x,cmd}^{new} = a_{x,cmd} \frac{a_{\max}}{\sqrt{a_{x,cmd}^2 + a_{y,cmd}^2}} \quad (46)$$

$$a_{y,cmd}^{new} = a_{y,cmd} \frac{a_{\max}}{\sqrt{a_{x,cmd}^2 + a_{y,cmd}^2}} \quad (47)$$

Bản chất của bước này là giới hạn độ mạnh của lệnh điều khiển vị trí. Nếu gia tốc ngang quá lớn, drone phải nghiêng quá nhiều, dễ gây mất ổn định.

1.2.8 Đổi từ [W] sang [B]

Gia tốc $a_{x,cmd}$ và $a_{y,cmd}$ đang nằm trong [W]. Tuy nhiên, roll và pitch lại là góc nghiêng theo thân drone.

Gọi yaw hiện tại của drone là:

$$\psi \quad (48)$$

Vector gia tốc trong [W] là:

$$\mathbf{a}^W = \begin{bmatrix} a_x \\ a_y \end{bmatrix} \quad (49)$$

Trục forward của drone trong [W] là:

$$\mathbf{e}_f = \begin{bmatrix} \cos \psi \\ \sin \psi \end{bmatrix} \quad (50)$$

Trục left của drone trong [W] là:

$$\mathbf{e}_l = \begin{bmatrix} -\sin \psi \\ \cos \psi \end{bmatrix} \quad (51)$$

Gia tốc theo hướng forward của drone là hình chiếu của vector gia tốc world lên trục forward:

$$a_f = \mathbf{a}^W \cdot \mathbf{e}_f \quad (52)$$

Suy ra:

$$a_f = a_x \cos \psi + a_y \sin \psi \quad (53)$$

Gia tốc theo hướng left của drone là:

$$a_l = \mathbf{a}^W \cdot \mathbf{e}_l \quad (54)$$

Suy ra:

$$a_l = -a_x \sin \psi + a_y \cos \psi \quad (55)$$

Trong đó:

- a_f : gia tốc mong muốn theo hướng tiến/lùi của drone,
- a_l : gia tốc mong muốn theo hướng trái/phải của drone.

1.2.9 Bản chất vật lý - nghiêng drone để tạo gia tốc ngang

Quadcopter tạo lực nâng tổng T theo trục thân của nó [2].

Khi drone nằm ngang, lực nâng hướng thẳng lên. Ở trạng thái hover:

$$T \approx mg \quad (56)$$

Khi drone nghiêng một góc α , lực nâng bị nghiêng theo. Khi đó lực nâng có thành phần đứng và thành phần ngang:

$$T_z = T \cos \alpha \quad (57)$$

$$T_h = T \sin \alpha \quad (58)$$

Để giữ độ cao gần như không đổi:

$$T \cos \alpha \approx mg \quad (59)$$

Theo phương ngang:

$$T \sin \alpha = ma_h \quad (60)$$

Chia hai phương trình:

$$\frac{T \sin \alpha}{T \cos \alpha} = \frac{ma_h}{mg} \quad (61)$$

Suy ra:

$$\tan \alpha = \frac{a_h}{g} \quad (62)$$

Do đó:

$$\alpha = \arctan \left(\frac{a_h}{g} \right) \quad (63)$$

Đây là công thức vật lý cơ bản để chuyển từ gia tốc ngang mong muốn sang góc nghiêng mong muốn.

1.2.10 Tính pitch target

Gọi gia tốc mong muốn theo hướng forward là:

$$a_f \quad (64)$$

Góc pitch mong muốn là:

$$\theta_d \quad (65)$$

Theo quan hệ vật lý:

$$\tan \theta_d = \frac{a_f}{g} \quad (66)$$

Suy ra:

$$\theta_d = \arctan \left(\frac{a_f}{g} \right) \quad (67)$$

Trong project, công thức được viết tương đương dưới dạng:

$$\theta_d = \text{atan2}(a_f, g) \quad (68)$$

Với góc nhỏ:

$$\theta_d \approx \frac{a_f}{g} \quad (69)$$

Trong đó:

$$\theta_d = \text{pitch_target} \quad (70)$$

1.2.11 Tính roll target

Gọi gia tốc mong muốn theo hướng left là:

$$a_l \quad (71)$$

Góc roll mong muốn là:

$$\phi_d \quad (72)$$

Theo quy ước dấu trong project:

$$\phi_d = -\text{atan2}(a_l, g) \quad (73)$$

Tương đương:

$$\phi_d = -\arctan \left(\frac{a_l}{g} \right) \quad (74)$$

Với góc nhỏ:

$$\phi_d \approx -\frac{a_l}{g} \quad (75)$$

Trong đó:

$$\phi_d = \text{roll_target} \quad (76)$$

Dấu âm xuất hiện do quy ước trục roll trong mô hình. Nếu sau này drone bay ngược hướng trái/phải, đây là một dấu cần kiểm tra.

1.2.12 Giới hạn góc nghiêng

Sau khi tính được ϕ_d và θ_d , project giới hạn:

$$-\alpha_{\max} \leq \phi_d \leq \alpha_{\max} \quad (77)$$

$$-\alpha_{\max} \leq \theta_d \leq \alpha_{\max} \quad (78)$$

Nếu:

$$\alpha_{\max} = 6.5^\circ \quad (79)$$

thì gia tốc ngang tối đa tương ứng xấp xỉ là:

$$a_{\max} = g \tan(\alpha_{\max}) \quad (80)$$

Thay số:

$$a_{\max} = 9.81 \tan(6.5^\circ) \quad (81)$$

$$a_{\max} \approx 1.12 \text{ m/s}^2 \quad (82)$$

Điều này cho thấy giới hạn góc nghiêng và giới hạn gia tốc ngang có liên hệ trực tiếp với nhau.

1.2.13 Công thức tổng hợp của Position Hold PID

Toàn bộ tầng Position Hold có thể viết tóm tắt như sau.

Sai số vị trí:

$$e_x = x_d - x \quad (83)$$

$$e_y = y_d - y \quad (84)$$

Sai số vận tốc:

$$e_{v_x} = v_{xd} - v_x \quad (85)$$

$$e_{v_y} = v_{yd} - v_y \quad (86)$$

Tích phân sai số:

$$I_x = \int e_x dt \quad (87)$$

$$I_y = \int e_y dt \quad (88)$$

PID tạo gia tốc ngang mong muốn:

$$a_x = K_{p_x} e_x + K_{d_x} e_{v_x} + K_{i_x} I_x \quad (89)$$

$$a_y = K_{p_y} e_y + K_{d_y} e_{v_y} + K_{i_y} I_y \quad (90)$$

Đổi sang hệ trục của drone:

$$a_f = a_x \cos \psi + a_y \sin \psi \quad (91)$$

$$a_l = -a_x \sin \psi + a_y \cos \psi \quad (92)$$

Đổi gia tốc thành góc nghiêng:

$$\theta_d = \text{atan2}(a_f, g) \quad (93)$$

$$\phi_d = -\text{atan2}(a_l, g) \quad (94)$$

1.3 Altitude Hold PID

Tầng altitude điều khiển độ cao z của drone. Khác với Position PID, tầng này không tạo góc nghiêng mà tạo PWM base cho cả bốn motor.

1.3.1 Sai số độ cao và vận tốc đứng

Gọi độ cao mong muốn là:

$$z_d \quad (95)$$

Độ cao hiện tại là:

$$z \quad (96)$$

Sai số độ cao:

$$e_z = z_d - z \quad (97)$$

Gọi vận tốc đứng mong muốn là v_{zd} và vận tốc đứng hiện tại là v_z . Sai số vận tốc đứng:

$$e_{v_z} = v_{zd} - v_z \quad (98)$$

Trong project, v_{zd} mặc định bằng 0, nghĩa là khi giữ độ cao thì drone không mong muốn tiếp tục đi lên hoặc đi xuống.

1.3.2 Tích phân độ cao

Thành phần tích phân của sai số độ cao:

$$I_z(k) = I_z(k-1) + e_z(k)\Delta t \quad (99)$$

Project giới hạn:

$$-I_{z,max} \leq I_z \leq I_{z,max} \quad (100)$$

Trong code:

$$I_{z,max} = 12.0 \quad (101)$$

Giới hạn này giúp tránh integral windup khi drone ở xa target hoặc khi PWM đã bị kẹp ở biên.

1.3.3 Output của Altitude PID

Altitude PID tạo ra lượng thay đổi PWM:

$$\Delta PWM_z = K_{p_z} e_z + K_{d_z} e_{v_z} + K_{i_z} I_z \quad (102)$$

Base PWM được tính bằng:

$$PWM_{base} = PWM_{hover} + \Delta PWM_z \quad (103)$$

Sau đó được giới hạn trong khoảng:

$$PWM_{min} \leq PWM_{base} \leq PWM_{max} \quad (104)$$

Với:

$$PWM_{min} = 1100, \quad PWM_{max} = 1940 \quad (105)$$

Do đó công thức đầy đủ trong code là:

$$PWM_{base} = clamp(PWM_{hover} + K_{p_z}e_z + K_{d_z}e_{v_z} + K_{i_z}I_z, 1100, 1940) \quad (106)$$

1.3.4 Ý nghĩa vật lý

PWM_{base} là mức ga chung cho bốn motor. Nếu drone thấp hơn target thì $e_z > 0$, khi đó ΔPWM_z tăng và tổng lực nâng tăng. Nếu drone cao hơn target thì $e_z < 0$, PWM nền giảm để drone hạ xuống.

Tầng altitude chỉ lo tổng lực nâng, còn việc phân bố lực giữa các motor để giữ roll, pitch, yaw sẽ do mixer xử lý.

1.4 Attitude Hold PID

Tầng attitude giữ roll và pitch của drone. Input của tầng này là góc mong muốn từ position controller:

$$\phi_d, \quad \theta_d \quad (107)$$

và góc hiện tại:

$$\phi, \quad \theta \quad (108)$$

Trong đó ϕ là roll và θ là pitch.

1.4.1 Sai số roll/pitch

Sai số roll:

$$e_\phi = wrap(\phi_d - \phi) \quad (109)$$

Sai số pitch:

$$e_\theta = wrap(\theta_d - \theta) \quad (110)$$

Hàm $wrap$ giữ sai số góc trong khoảng $[-\pi, \pi]$.

1.4.2 Thành phần vi phân dùng vận tốc góc

Trong code, thành phần D của attitude PID không tính bằng:

$$\frac{de_\phi}{dt}, \quad \frac{de_\theta}{dt} \quad (111)$$

mà dùng trực tiếp vận tốc góc body:

$$p = \dot{\phi}, \quad q = \dot{\theta} \quad (112)$$

Nếu dùng PID chuẩn, thành phần D là:

$$K_{d\phi} \frac{de_\phi}{dt} = K_{d\phi} \left(\frac{d\phi_d}{dt} - \frac{d\phi}{dt} \right) = K_{d\phi} (\dot{\phi}_d - \dot{\phi}) \quad (113)$$

Nếu roll target thay đổi chậm: $\dot{\phi}_d \approx 0$ và $p \approx \dot{\phi}$
 Vì vậy:

$$K_{d\phi} \frac{de_\phi}{dt} \approx -K_{d\phi} \dot{\phi}_d = -K_{d\phi} p \quad (114)$$

Tương tự với pitch, đó là lí do công thức có dấu trừ:

$$\Delta PWM_\phi = K_{p_\phi} e_\phi + K_{i_\phi} I_\phi - K_{d_\phi} p \quad (115)$$

$$\Delta PWM_\theta = K_{p_\theta} e_\theta + K_{i_\theta} I_\theta - K_{d_\theta} q \quad (116)$$

Khi drone đang quay nhanh theo một chiều, thành phần $-K_d p$ hoặc $-K_d q$ tạo lực hãm để giảm dao động.

1.4.3 Tích phân và giới hạn output

Tích phân roll/pitch:

$$I_\phi(k) = I_\phi(k-1) + e_\phi(k) \Delta t \quad (117)$$

$$I_\theta(k) = I_\theta(k-1) + e_\theta(k) \Delta t \quad (118)$$

Trong code:

$$-1.0 \leq I_\phi, I_\theta \leq 1.0 \quad (119)$$

Output PWM correction cũng được kẹp:

$$-260 \leq \Delta PWM_\phi, \Delta PWM_\theta \leq 260 \quad (120)$$

Các giới hạn này làm cho attitude controller không thể yêu cầu mixer tạo ra lệnh quá mạnh so với khả năng motor.

1.5 Yaw Hold PID

Yaw controller giữ hướng ψ của drone. Trong project, yaw controller có thể bật/tắt bằng biến `enabled`.

1.5.1 Sai số yaw

Yaw target:

$$\psi_d \quad (121)$$

Yaw hiện tại:

$$\psi \quad (122)$$

Sai số yaw:

$$e_\psi = \text{wrap}(\psi_d - \psi) \quad (123)$$

Việc wrap góc đặc biệt quan trọng với yaw vì yaw có thể đi qua biên $-\pi$ và π .

1.5.2 Công thức yaw correction

Gọi yaw rate là:

$$r = \dot{\psi} \quad (124)$$

Tích phân yaw:

$$I_\psi(k) = I_\psi(k-1) + e_\psi(k)\Delta t \quad (125)$$

Output của yaw PID là PWM correction:

$$\Delta PWM_\psi = K_{p_\psi} e_\psi + K_{i_\psi} I_\psi - K_{d_\psi} r \quad (126)$$

Trong code, output này không phải torque trực tiếp. Nó là lượng PWM sẽ được mixer cộng/trừ lên các motor để tạo chênh lệch torque phản lực.

Giới hạn tích phân:

$$-0.8 \leq I_\psi \leq 0.8 \quad (127)$$

Giới hạn output:

$$-120 \leq \Delta PWM_\psi \leq 120 \quad (128)$$

1.5.3 Vai trò của yaw correction trong quadcopter

Roll và pitch được tạo chủ yếu bằng chênh lệch lực nâng giữa các motor. Yaw được tạo bằng chênh lệch torque phản lực của các cánh quạt quay ngược chiều nhau. Vì vậy mixer sẽ cộng ΔPWM_ψ vào nhóm motor này và trừ khỏi nhóm motor kia.

Nếu yaw hold tắt, project đặt:

$$\Delta PWM_\psi = 0 \quad (129)$$

Khi đó mixer chỉ còn giữ độ cao, roll và pitch.

1.6 Quad X PWM Mixer

Mixer là tầng nhận các lệnh PWM trừu tượng và tạo PWM cụ thể cho từng motor. Cấu hình của drone là quad X với thứ tự motor:

Motor	Vị trí trên frame	Tọa độ tương đối
1	front-left	$(+arm, +arm, z_m)$
2	front-right	$(+arm, -arm, z_m)$
3	rear-right	$(-arm, -arm, z_m)$
4	rear-left	$(-arm, +arm, z_m)$

Trong code:

$$arm = 0.159, \quad z_m = 0.04 \quad (130)$$

1.6.1 Công thức mixer

Mixer nhận:

$$PWM_{base}, \quad \Delta PWM_{\phi}, \quad \Delta PWM_{\theta}, \quad \Delta PWM_{\psi} \quad (131)$$

và tính:

$$PWM_1 = PWM_{base} + \Delta PWM_{\phi} - \Delta PWM_{\theta} - \Delta PWM_{\psi} \quad (132)$$

$$PWM_2 = PWM_{base} - \Delta PWM_{\phi} - \Delta PWM_{\theta} + \Delta PWM_{\psi} \quad (133)$$

$$PWM_3 = PWM_{base} - \Delta PWM_{\phi} + \Delta PWM_{\theta} - \Delta PWM_{\psi} \quad (134)$$

$$PWM_4 = PWM_{base} + \Delta PWM_{\phi} + \Delta PWM_{\theta} + \Delta PWM_{\psi} \quad (135)$$

Có thể được viết dưới dạng ma trận:

$$\begin{bmatrix} PWM_1 \\ PWM_2 \\ PWM_3 \\ PWM_4 \end{bmatrix} = PWM_{base} \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} + \Delta PWM_{\phi} \begin{bmatrix} +1 \\ -1 \\ -1 \\ +1 \end{bmatrix} + \Delta PWM_{\theta} \begin{bmatrix} -1 \\ -1 \\ +1 \\ +1 \end{bmatrix} + \Delta PWM_{\psi} \begin{bmatrix} -1 \\ +1 \\ -1 \\ +1 \end{bmatrix} \quad (136)$$

Sau đó từng PWM được giới hạn:

$$1100 \leq PWM_i \leq 1940 \quad (137)$$

1.6.2 Ý nghĩa dấu trong mixer

Các dấu cộng/trừ trong mixer không được chọn ngẫu nhiên. Chúng xuất phát từ vị trí của từng motor trên thân drone và chiều quay của từng cánh quạt.

Ý tưởng chính của mixer là:

$$PWM_{base} \rightarrow \text{tạo lực nâng chung} \quad (138)$$

$$\Delta PWM_\phi, \Delta PWM_\theta, \Delta PWM_\psi \rightarrow \text{tạo chênh lệch lực để sinh moment quay} \quad (139)$$

Trong đó:

- ΔPWM_ϕ : hiệu chỉnh roll quanh trục x .
- ΔPWM_θ : hiệu chỉnh pitch quanh trục y .
- ΔPWM_ψ : hiệu chỉnh yaw quanh trục z .

1.6.2.1 Quy ước hệ trục và vị trí motor

Trong project này, hệ trục body của drone được quy ước:

$$+x : \text{phía trước}, \quad +y : \text{phía trái}, \quad +z : \text{hướng lên} \quad (140)$$

Bốn motor được đặt tại các vị trí:

$$M_1 : (+x, +y) \quad (141)$$

$$M_2 : (+x, -y) \quad (142)$$

$$M_3 : (-x, -y) \quad (143)$$

$$M_4 : (-x, +y) \quad (144)$$

Do đó:

- M_1 và M_4 nằm bên trái drone vì có $y > 0$.
- M_2 và M_3 nằm bên phải drone vì có $y < 0$.
- M_1 và M_2 nằm phía trước vì có $x > 0$.
- M_3 và M_4 nằm phía sau vì có $x < 0$.

1.6.2.2 Vì sao chênh lệch lực tạo ra moment?

Mỗi motor tạo ra một lực nâng F_i theo hướng lên. Vì motor không nằm ở tâm drone, lực nâng này sẽ tạo moment quay quanh tâm drone.

Công thức moment tổng quát là:

$$\boldsymbol{\tau} = \mathbf{r} \times \mathbf{F} \quad (145)$$

Trong đó:

- $\mathbf{r}$ là vector từ tâm drone đến vị trí motor.
- $\mathbf{F}$ là lực nâng do motor tạo ra.
- $\boldsymbol{\tau}$ là moment làm drone quay.

Nếu lực đặt đúng tại tâm drone thì $\mathbf{r} = 0$, do đó không tạo moment. Ngược lại, nếu lực đặt lệch khỏi tâm, tức là $\mathbf{r} \neq 0$, lực đó sẽ làm drone quay.

Với motor thứ i , ta có thể viết:

$$\mathbf{r}_i = \begin{bmatrix} x_i \\ y_i \\ 0 \end{bmatrix}, \quad \mathbf{F}_i = \begin{bmatrix} 0 \\ 0 \\ F_i \end{bmatrix} \quad (146)$$

Moment do motor đó tạo ra là:

$$\boldsymbol{\tau}_i = \mathbf{r}_i \times \mathbf{F}_i = \begin{bmatrix} y_i F_i \\ -x_i F_i \\ 0 \end{bmatrix} \quad (147)$$

Do đó:

$$\tau_x = y_i F_i \quad (148)$$

$$\tau_y = -x_i F_i \quad (149)$$

Nghĩa là:

- Moment roll quanh trục x phụ thuộc vào $y_i F_i$.
- Moment pitch quanh trục y phụ thuộc vào $-x_i F_i$.

1.6.2.3 Ý nghĩa của PWM nền

PWM_{base} là PWM nền được cộng đều vào cả bốn motor:

$$PWM_1 = PWM_2 = PWM_3 = PWM_4 = PWM_{base} \quad (150)$$

Khi bốn motor nhận PWM gần như bằng nhau, chúng tạo lực nâng gần như bằng nhau. Khi đó các moment roll và pitch triệt tiêu nhau, drone chủ yếu hover, bay lên hoặc bay xuống. Có thể hiểu:

$$PWM_{base} \rightarrow \text{điều khiển tổng lực nâng} \quad (151)$$

1.6.2.4 Ý nghĩa chung của các delta PWM

Các thành phần delta PWM không nhằm tăng đều cả bốn motor, mà nhằm tăng motor này và giảm motor khác để tạo mất cân bằng lực.

Dạng tổng quát của mixer là:

$$\begin{bmatrix} PWM_1 \\ PWM_2 \\ PWM_3 \\ PWM_4 \end{bmatrix} = PWM_{base} \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} + \Delta PWM_\phi \begin{bmatrix} +1 \\ -1 \\ -1 \\ +1 \end{bmatrix} + \Delta PWM_\theta \begin{bmatrix} -1 \\ -1 \\ +1 \\ +1 \end{bmatrix} + \Delta PWM_\psi \begin{bmatrix} -1 \\ +1 \\ -1 \\ +1 \end{bmatrix} \quad (152)$$

Các vector dấu của delta PWM đều có tổng bằng 0:

$$(+1) + (-1) + (-1) + (+1) = 0 \quad (153)$$

$$(-1) + (-1) + (+1) + (+1) = 0 \quad (154)$$

$$(-1) + (+1) + (-1) + (+1) = 0 \quad (155)$$

Vì vậy, về ý tưởng, các delta PWM không làm thay đổi nhiều tổng lực nâng. Chúng chủ yếu tạo chênh lệch lực giữa các motor để tạo moment quay.

1.6.2.5 Roll correction tạo moment roll

Roll là chuyển động quay quanh trục x . Moment roll tổng do bốn motor tạo ra có thể viết là:

$$\tau_x = \sum_{i=1}^4 y_i F_i \quad (156)$$

Trong đó y_i là vị trí của motor theo trục y , còn F_i là lực nâng của motor đó. Với layout hiện tại:

$$y_1 = +l, \quad y_4 = +l \quad (157)$$

$$y_2 = -l, \quad y_3 = -l \quad (158)$$

Nếu $\Delta PWM_\phi > 0$, mixer chọn:

$$M_1 : +, \quad M_2 : -, \quad M_3 : -, \quad M_4 : + \quad (159)$$

Nghĩa là:

- M_1 và M_4 tăng lực nâng.
- M_2 và M_3 giảm lực nâng.

Ta có thể viết gần đúng:

$$F_1 = F_0 + \Delta F \quad (160)$$

$$F_4 = F_0 + \Delta F \quad (161)$$

$$F_2 = F_0 - \Delta F \quad (162)$$

$$F_3 = F_0 - \Delta F \quad (163)$$

Khi đó:

$$\begin{aligned} \tau_x &= y_1 F_1 + y_2 F_2 + y_3 F_3 + y_4 F_4 \\ &= l(F_0 + \Delta F) - l(F_0 - \Delta F) - l(F_0 - \Delta F) + l(F_0 + \Delta F) \end{aligned} \quad (164)$$

Các thành phần F_0 triệt tiêu nhau, còn lại:

$$\tau_x = 4l\Delta F \quad (165)$$

Vì vậy, nếu $\Delta F > 0$ thì:

$$\tau_x > 0 \quad (166)$$

Nghĩa là chênh lệch lực trái-phải tạo ra moment roll dương.

1.6.2.6 Vì sao roll correction không làm đổi nhiều tổng lực nâng?

Tổng lực nâng của bốn motor là:

$$F_{total} = F_1 + F_2 + F_3 + F_4 \quad (167)$$

Thay các lực sau khi roll correction:

$$F_{total} = (F_0 + \Delta F) + (F_0 - \Delta F) + (F_0 - \Delta F) + (F_0 + \Delta F) \quad (168)$$

Các thành phần ΔF triệt tiêu:

$$+\Delta F - \Delta F - \Delta F + \Delta F = 0 \quad (169)$$

Do đó:

$$F_{total} = 4F_0 \quad (170)$$

Vì vậy roll correction gần như không làm đổi tổng lực nâng. Nó chủ yếu làm lực bên trái lớn hơn bên phải, từ đó tạo moment roll.

1.6.2.7 Pitch correction tạo moment pitch

Pitch là chuyển động quay quanh trục y . Moment pitch tổng có dạng:

$$\tau_y = \sum_{i=1}^4 -x_i F_i \quad (171)$$

Motor phía trước có $x > 0$ là M_1 và M_2 . Motor phía sau có $x < 0$ là M_3 và M_4 . Nếu $\Delta PWM_\theta > 0$, mixer chọn:

$$M_1 : -, \quad M_2 : -, \quad M_3 : +, \quad M_4 : + \quad (172)$$

Nghĩa là:

- M_1 và M_2 giảm lực nâng.
- M_3 và M_4 tăng lực nâng.

Khi đó phía sau đẩy mạnh hơn phía trước. Vì phía sau có $x < 0$, theo công thức:

$$\tau_y = \sum_{i=1}^4 -x_i F_i \quad (173)$$

sự tăng lực ở phía sau sẽ tạo moment pitch dương theo quy ước của project.

Do đó vector dấu của pitch correction là:

$$\Delta PWM_\theta \begin{bmatrix} -1 \\ -1 \\ +1 \\ +1 \end{bmatrix} \quad (174)$$

1.6.2.8 Yaw correction tạo moment yaw

Yaw là chuyển động quay quanh trục z . Phần yaw khó hơn roll và pitch vì yaw không được tạo chủ yếu bằng cách tạo chênh lệch lực nâng trái-phải hoặc trước-sau. Yaw chủ yếu được tạo bởi torque phản lực của cánh quạt.

Khi một motor làm cánh quạt quay, motor tác dụng một torque lên cánh quạt. Theo định luật III Newton, cánh quạt cũng tác dụng ngược lại một torque lên thân drone. Vì vậy, khi cánh quạt quay một chiều, thân drone có xu hướng quay chiều ngược lại.

Đây là nguồn gốc của yaw torque.

Trong project, chiều quay motor được quy ước bởi vector:

$$d = \begin{bmatrix} +1 \\ -1 \\ +1 \\ -1 \end{bmatrix} \quad (175)$$

Nghĩa là M_1 và M_3 quay cùng một chiều, còn M_2 và M_4 quay chiều ngược lại.

Gọi Q_i là reaction torque của motor thứ i . Torque yaw tổng tác dụng lên body được mô hình hóa bởi:

$$M_z = \sum_{i=1}^4 -d_i Q_i \quad (176)$$

Viết tường minh:

$$M_z = -d_1 Q_1 - d_2 Q_2 - d_3 Q_3 - d_4 Q_4 \quad (177)$$

Vì:

$$d_1 = +1, \quad d_2 = -1, \quad d_3 = +1, \quad d_4 = -1 \quad (178)$$

nên:

$$\begin{aligned} M_z &= -(+1)Q_1 - (-1)Q_2 - (+1)Q_3 - (-1)Q_4 \\ &= -Q_1 + Q_2 - Q_3 + Q_4 \end{aligned} \quad (179)$$

Khi hover đều, bốn motor tạo reaction torque gần như bằng nhau:

$$Q_1 = Q_2 = Q_3 = Q_4 = Q_0 \quad (180)$$

Thay vào công thức trên:

$$\begin{aligned} M_z &= -Q_0 + Q_0 - Q_0 + Q_0 \\ &= 0 \end{aligned} \quad (181)$$

Vì vậy, khi bốn motor chạy đều, các reaction torque quanh trục z triệt tiêu nhau và drone không tự quay yaw.

Tạo yaw moment dương

Từ công thức:

$$M_z = -Q_1 + Q_2 - Q_3 + Q_4 \quad (182)$$

ta thấy Q_2 và Q_4 đóng góp dấu dương vào M_z , còn Q_1 và Q_3 đóng góp dấu âm vào M_z .

Do đó, để tạo yaw moment dương:

$$M_z > 0 \quad (183)$$

ta cần tăng reaction torque của nhóm M_2, M_4 và giảm reaction torque của nhóm M_1, M_3 :

$$Q_2 \uparrow, \quad Q_4 \uparrow \quad (184)$$

$$Q_1 \downarrow, \quad Q_3 \downarrow \quad (185)$$

Vì reaction torque Q_i tăng khi PWM motor tăng, điều này tương ứng với:

$$PWM_2 \uparrow, \quad PWM_4 \uparrow \quad (186)$$

$$PWM_1 \downarrow, \quad PWM_3 \downarrow \quad (187)$$

Do đó, nếu $\Delta PWM_\psi > 0$, mixer chọn:

$$M_1 : -, \quad M_2 : +, \quad M_3 : -, \quad M_4 : + \quad (188)$$

Hay viết dưới dạng vector dấu:

$$\Delta PWM_\psi \begin{bmatrix} -1 \\ +1 \\ -1 \\ +1 \end{bmatrix} \quad (189)$$

Chứng minh bằng công thức Giả sử ban đầu reaction torque của bốn motor bằng nhau:

$$Q_1 = Q_2 = Q_3 = Q_4 = Q_0 \quad (190)$$

Sau khi áp dụng yaw correction dương:

$$Q_1 = Q_0 - \Delta Q \quad (191)$$

$$Q_2 = Q_0 + \Delta Q \quad (192)$$

$$Q_3 = Q_0 - \Delta Q \quad (193)$$

$$Q_4 = Q_0 + \Delta Q \quad (194)$$

Thay vào công thức:

$$M_z = -Q_1 + Q_2 - Q_3 + Q_4 \quad (195)$$

ta được:

$$M_z = -(Q_0 - \Delta Q) + (Q_0 + \Delta Q) - (Q_0 - \Delta Q) + (Q_0 + \Delta Q) \quad (196)$$

Khai triển:

$$M_z = -Q_0 + \Delta Q + Q_0 + \Delta Q - Q_0 + \Delta Q + Q_0 + \Delta Q \quad (197)$$

Các thành phần Q_0 triệt tiêu nhau:

$$-Q_0 + Q_0 - Q_0 + Q_0 = 0 \quad (198)$$

Còn lại:

$$M_z = 4\Delta Q \quad (199)$$

Vì vậy, nếu:

$$\Delta Q > 0 \quad (200)$$

thì:

$$M_z > 0 \quad (201)$$

Nghĩa là drone tạo được yaw moment dương.

Vì sao yaw correction không làm đổi nhiều tổng lực nâng? Vector dấu của yaw correction là:

$$\begin{bmatrix} -1 \\ +1 \\ -1 \\ +1 \end{bmatrix} \quad (202)$$

Tổng các phần tử của vector này bằng 0:

$$(-1) + (+1) + (-1) + (+1) = 0 \quad (203)$$

Điều này có nghĩa là mixer giảm hai motor và tăng hai motor. Vì vậy, về ý tưởng, tổng lực nâng không thay đổi nhiều. Tuy nhiên, do hai nhóm motor quay ngược chiều nhau, reaction torque không còn cân bằng nữa, nên drone sẽ quay quanh trục yaw.

So sánh yaw với roll và pitch Roll và pitch được tạo chủ yếu bởi chênh lệch lực nâng theo vị trí motor:

$$\tau_x = \sum_{i=1}^4 y_i F_i \quad (204)$$

$$\tau_y = \sum_{i=1}^4 -x_i F_i \quad (205)$$

Nghĩa là roll và pitch phụ thuộc vào motor nằm bên trái-phải hoặc trước-sau.

Yaw thì khác. Yaw chủ yếu phụ thuộc vào chiều quay của cánh quạt:

$$M_z = \sum_{i=1}^4 -d_i Q_i \quad (206)$$

Do đó, yaw correction không chia motor theo trái-phải hoặc trước-sau, mà chia theo hai nhóm motor quay ngược chiều:

$$M_1, M_3 \quad (207)$$

và:

$$M_2, M_4 \quad (208)$$

Cách nhớ nhanh Trong project:

$$d = \begin{bmatrix} +1 \\ -1 \\ +1 \\ -1 \end{bmatrix} \quad (209)$$

và:

$$M_z = \sum_{i=1}^4 -d_i Q_i \quad (210)$$

Do có dấu trừ trước d_i , ta có:

$$d_i = +1 \Rightarrow \text{đóng góp yaw âm} \quad (211)$$

$$d_i = -1 \Rightarrow \text{đóng góp yaw dương} \quad (212)$$

Vì vậy, muốn tạo yaw moment dương thì giảm nhóm $d_i = +1$ và tăng nhóm $d_i = -1$:

$$M_1 : -, \quad M_2 : +, \quad M_3 : -, \quad M_4 : + \quad (213)$$

Do đó:

$$\Delta PWM_\psi \begin{bmatrix} -1 \\ +1 \\ -1 \\ +1 \end{bmatrix} \quad (214)$$

Tóm tắt Motor quay cánh quạt sẽ tạo reaction torque lên thân drone. Hai motor M_1, M_3 tạo yaw torque một chiều, còn hai motor M_2, M_4 tạo yaw torque chiều ngược lại. Khi hover đều, hai chiều torque này triệt tiêu nhau. Muốn drone quay yaw, mixer tăng PWM của một nhóm và giảm PWM của nhóm còn lại. Nhờ vậy tổng lực nâng gần như giữ nguyên, nhưng torque yaw không còn cân bằng, nên drone quay quanh trục z .

1.7 Motor Model

Motor model mô phỏng đơn giản ESC, motor BLDC và propeller. Input là PWM, output là lực đẩy, dòng điện, RPM và torque phản lực.

1.7.1 Đổi PWM sang throttle

PWM được đưa về tỉ lệ ga u :

$$u = \frac{PWM - PWM_{min}}{PWM_{max} - PWM_{min}} \quad (215)$$

Với:

$$0 \leq u \leq 1 \quad (216)$$

Điện áp hiệu dụng đưa vào motor:

$$V_m = uV_{bat} \quad (217)$$

Trong code:

$$V_{bat} = 14.8 \text{ V} \quad (218)$$

1.7.2 Dòng điện và torque motor

Hằng số back-EMF được tính từ KV:

$$K_e = \frac{60}{2\pi KV} \quad (219)$$

Code dùng:

$$K_t = K_e \quad (220)$$

Dòng điện motor:

$$I = \frac{V_m - K_e\omega}{R_m} \quad (221)$$

Dòng điện được giới hạn:

$$0 \leq I \leq I_{max} \quad (222)$$

Torque motor:

$$\tau_m = K_t \max(I - I_0, 0) \quad (223)$$

Trong đó I_0 là dòng không tải.

1.7.3 Động lực học rotor

Torque tải khí động của propeller:

$$\tau_{load} = k_q\omega^2 \quad (224)$$

Gia tốc góc rotor:

$$\dot{\omega} = \frac{\tau_m - \tau_{load}}{J} \quad (225)$$

Cập nhật rời rạc:

$$\omega(k) = \omega(k-1) + \dot{\omega}\Delta t \quad (226)$$

Sau đó ω được kẹp trong khoảng:

$$0 \leq \omega \leq \omega_{max} \quad (227)$$

Trong code:

$$\omega_{max} = 1800 \text{ rad/s} \quad (228)$$

1.7.4 Lực đẩy, torque phản lực và RPM

Lực đẩy propeller:

$$T = k_f \omega^2 \quad (229)$$

Torque phản lực:

$$Q = k_q \omega^2 \quad (230)$$

RPM:

$$RPM = \frac{\omega 60}{2\pi} \quad (231)$$

Các tham số mặc định của motor model:

Tham số	Giá trị
KV	1100
R_m	0.073Ω
I_0	0.9 A
I_{max}	30 A
J	2.5×10^{-5}
k_f	4.0×10^{-6}
k_q	1.0×10^{-7}

1.8 Apply lực vào Isaac Sim

Sau khi motor model trả về lực đẩy T_i , controller apply lực lên rigid body `base_link` tại vị trí của từng motor:

$$\mathbf{F}_i = \begin{bmatrix} 0 \\ 0 \\ T_i \end{bmatrix} \quad (232)$$

Vì lực được đặt lệch khỏi tâm drone, Isaac Sim tự sinh moment roll/pitch tương ứng. Đây là lý do mixer không cần apply moment roll/pitch trực tiếp; nó chỉ cần tạo chênh lệch lực motor.

1.8.1 Torque phản lực yaw

Mỗi propeller tạo torque phản lực Q_i . Với chiều quay motor:

$$d = [1, -1, 1, -1] \quad (233)$$

Torque yaw tổng lên body được tính:

$$M_z = \sum_{i=1}^4 -d_i Q_i \quad (234)$$

Trong code, torque yaw này được đổi thành một cặp lực ngang đặt tại hai điểm đối xứng:

$$F_y = \frac{M_z}{2arm} \quad (235)$$

Một lực $+F_y$ đặt tại $x = +arm$, lực còn lại $-F_y$ đặt tại $x = -arm$. Hai lực này triệt tiêu tổng lực tuyến tính nhưng tạo moment quanh trục z .

1.8.2 Kết nối giữa motor model và simulator

Mối quan hệ cuối cùng giữa PWM và chuyển động drone là:

$$PWM_i \longrightarrow \omega_i \longrightarrow T_i, Q_i \longrightarrow IsaacSim \longrightarrow x, y, z, \phi, \theta, \psi \quad (236)$$

Như vậy, controller không trực tiếp thay đổi pose của drone. Controller chỉ apply lực vật lý, còn chuyển động thật do engine physics của Isaac Sim tích phân.

1.9 Cách xác định output của một bộ PID

Một bộ PID không tự quyết định output là gia tốc, lực, moment hay PWM. Người thiết kế quyết định output dựa trên câu hỏi: "Tầng sau của PID cần nhận đại lượng gì?"

Có thể viết nguyên tắc này dưới dạng:

$$\boxed{\text{Đơn vị output của PID} = \text{Đơn vị input mà tầng sau cần}} \quad (237)$$

Ví dụ, nếu tầng sau cần gia tốc:

$$u = a \quad (238)$$

Nếu tầng sau cần lực:

$$u = F \quad (239)$$

Nếu tầng sau cần moment:

$$u = \tau \quad (240)$$

Nếu tầng sau cần PWM:

$$u = PWM \quad (241)$$

1.9.1 Output của các tầng PID trong project

1.9.1.1 Position PID

Tầng Position PID nhận sai số vị trí và biến nó thành lệnh gia tốc/góc nghiêng mong muốn [3]:

$$e_x, \quad e_y \quad (242)$$

và tạo gia tốc ngang mong muốn:

$$a_x, \quad a_y \quad (243)$$

Sau đó đổi thành góc nghiêng mong muốn:

$$a_x, \quad a_y \longrightarrow \phi_d, \quad \theta_d \quad (244)$$

Vì vậy:

$$\boxed{PositionPID \longrightarrow roll_target, \quad pitch_target} \quad (245)$$

1.9.1.2 Altitude PID

Tầng Altitude PID điều khiển độ cao z .

Sai số độ cao:

$$e_z = z_d - z \quad (246)$$

Sai số vận tốc đứng:

$$e_{v_z} = v_{zd} - v_z \quad (247)$$

Trong project, output của tầng này là PWM nên:

$$PWM_{base} = PWM_{hover} + K_{p_z} e_z + K_{d_z} e_{v_z} + K_{i_z} \int e_z dt \quad (248)$$

Vì vậy:

$$\boxed{AltitudePID \longrightarrow PWM_{base}} \quad (249)$$

1.9.1.3 Attitude PID

Tầng Attitude PID nhận góc mong muốn:

$$\phi_d, \quad \theta_d \quad (250)$$

và góc hiện tại:

$$\phi, \quad \theta \quad (251)$$

Sai số roll:

$$e_\phi = \phi_d - \phi \quad (252)$$

Sai số pitch:

$$e_\theta = \theta_d - \theta \quad (253)$$

Trong project, output của tầng này là PWM correction:

$$\Delta PWM_\phi = K_{p_\phi} e_\phi + K_{i_\phi} \int e_\phi dt - K_{d_\phi} p \quad (254)$$

$$\Delta PWM_\theta = K_{p_\theta} e_\theta + K_{i_\theta} \int e_\theta dt - K_{d_\theta} q \quad (255)$$

Trong đó:

$$p = \dot{\phi} \quad (256)$$

$$q = \dot{\theta} \quad (257)$$

Vì vậy:

$$\boxed{AttitudePID \longrightarrow \Delta PWM_\phi, \Delta PWM_\theta} \quad (258)$$

1.9.1.4 Yaw PID

Sai số yaw:

$$e_\psi = \text{wrap}(\psi_d - \psi) \quad (259)$$

Output của yaw PID là PWM correction theo yaw:

$$\Delta PWM_\psi = K_{p_\psi} e_\psi + K_{i_\psi} \int e_\psi dt - K_{d_\psi} r \quad (260)$$

Trong đó:

$$r = \dot{\psi} \quad (261)$$

Vì vậy:

$$\boxed{YawPID \longrightarrow \Delta PWM_{\psi}} \quad (262)$$

1.9.2 Mixer

Mixer nhận các đại lượng:

$$PWM_{base}, \quad \Delta PWM_{\phi}, \quad \Delta PWM_{\theta}, \quad \Delta PWM_{\psi} \quad (263)$$

và tạo PWM cho từng motor:

$$PWM_1 = PWM_{base} + \Delta PWM_{\phi} - \Delta PWM_{\theta} - \Delta PWM_{\psi} \quad (264)$$

$$PWM_2 = PWM_{base} - \Delta PWM_{\phi} - \Delta PWM_{\theta} + \Delta PWM_{\psi} \quad (265)$$

$$PWM_3 = PWM_{base} - \Delta PWM_{\phi} + \Delta PWM_{\theta} - \Delta PWM_{\psi} \quad (266)$$

$$PWM_4 = PWM_{base} + \Delta PWM_{\phi} + \Delta PWM_{\theta} + \Delta PWM_{\psi} \quad (267)$$

Vì vậy:

$$\boxed{Mixer \longrightarrow PWM_1, PWM_2, PWM_3, PWM_4} \quad (268)$$

1.9.3 Tóm tắt nguyên tắc chọn output PID

Nguyên tắc chung:

$$\boxed{PID \text{ output phụ thuộc vào tầng sau cần gì}} \quad (269)$$

Trong project hiện tại:

$$PositionPID \longrightarrow a_x, a_y \longrightarrow \phi_d, \theta_d \quad (270)$$

$$AltitudePID \longrightarrow PWM_{base} \quad (271)$$

$$AttitudePID \longrightarrow \Delta PWM_{\phi}, \Delta PWM_{\theta} \quad (272)$$

$$YawPID \longrightarrow \Delta PWM_{\psi} \quad (273)$$

$$Mixer \longrightarrow PWM_1, PWM_2, PWM_3, PWM_4 \quad (274)$$

$$MotorModel \longrightarrow T_1, T_2, T_3, T_4 \quad (275)$$

Do đó, trong project này:

- Position PID tạo gia tốc rồi đổi thành góc nghiêng mong muốn.

- Altitude PID tạo PWM nền.
- Attitude PID tạo PWM hiệu chỉnh roll/pitch.
- Yaw PID tạo PWM hiệu chỉnh yaw.
- Mixer tạo PWM cho từng motor.
- Motor model biến PWM thành lực nâng.

1.10 Propeller Spinner

File `propeller_spinner.py` không phải là PID controller, nhưng giúp cánh quạt trong Isaac Sim quay theo RPM của motor model. Module này dùng `dynamic_control` để tìm các DOF:

`propeller_1_joint, propeller_2_joint, propeller_3_joint, propeller_4_joint` (276)

Nếu tìm được DOF, code cấu hình velocity drive và đặt target velocity cho từng cánh quạt.

1.10.1 Đổi RPM sang vận tốc joint

Với motor thứ i , RPM từ motor model được đổi sang rad/s:

$$\omega_i = RPM_i \frac{2\pi}{60} \quad (277)$$

Lệnh vận tốc joint:

$$\omega_{cmd,i} = d_i s \omega_i \quad (278)$$

Trong đó:

- d_i là chiều quay motor, mặc định $[1, -1, 1, -1]$.
- s là hệ số hiển thị, mặc định 0.7.

Việc nhân với s giúp tốc độ quay hiển thị không quá cực đoan so với mô phỏng. Phần này chủ yếu phục vụ trực quan, không phải nguồn lực chính tác động lên drone. Lực chính vẫn đến từ `MotorModel` và `apply_body_force`.

1.11 Tracking Logger

File `tracking_logger.py` ghi log trạng thái drone và target ra CSV. Logger được bật bằng:

$$start_tracking_log(csv_path, sample_period) \quad (279)$$

Mặc định:

$$sample_period = 0.02 \text{ s} \quad (280)$$

Tức là tần số ghi log tối đa khoảng:

$$50 \text{ Hz} \quad (281)$$

Các đại lượng được ghi CSV log gồm các nhóm dữ liệu:

- Thời gian mô phỏng: `time`.
- Vị trí và target: x, x_d, y, y_d, z, z_d .
- Roll, pitch, yaw theo độ.
- Roll, pitch, yaw target theo độ.

- Roll, pitch, yaw theo radian.
- Roll, pitch, yaw target theo radian.

Các dữ liệu này đủ để vẽ đồ thị tracking, kiểm tra overshoot, kiểm tra sai số xác lập và so sánh target với trạng thái thật.

1.12 Compatibility Mixin

File `attitude_hold_compat.py` chứa `AttitudeHoldCompatibilityMixin`. Module này không tạo luật điều khiển mới. Vai trò của nó là giữ lại API cũ sau khi code được tách thành nhiều controller nhỏ.

Ví dụ, khi code bên ngoài truy cập:

$$kp_x \quad (282)$$

thì property trong mixin thực chất đọc:

$$position_controller.kp_x \quad (283)$$

Tương tự:

$$z_target \leftrightarrow altitude_controller.z_target \quad (284)$$

$$yaw_target \leftrightarrow yaw_controller.yaw_target \quad (285)$$

$$roll_target, pitch_target \leftrightarrow attitude_controller \quad (286)$$

Cách làm này giúp những script cũ vẫn chạy được, đồng thời code mới vẫn có cấu trúc rõ ràng hơn.

1.13 Tóm tắt toàn bộ hệ thống

Toàn bộ controller có thể tóm tắt bằng chuỗi biến đổi:

$$(x_d, y_d, z_d, \psi_d) - (x, y, z, \psi) \longrightarrow PID \longrightarrow PWM_i \longrightarrow T_i, Q_i \longrightarrow \text{Isaac Sim physics} \quad (287)$$

Trong đó:

- Position PID biến sai số x, y thành góc nghiêng mong muốn.
- Altitude PID biến sai số z thành PWM nền.
- Attitude PID biến sai số roll/pitch thành PWM correction.
- Yaw PID biến sai số yaw thành PWM correction.
- Mixer tạo PWM cho từng motor theo cấu hình quad X.
- Motor model biến PWM thành lực đẩy và torque phản lực.
- Isaac Sim nhận lực vật lý và cập nhật trạng thái drone.

Điểm quan trọng nhất là mỗi PID chỉ tạo output mà tầng sau cần. Nhờ vậy hệ điều khiển dễ đọc, dễ chỉnh tham số và dễ debug: khi drone lỗi độ cao thì kiểm tra altitude PID; khi nghiêng sai thì kiểm tra attitude PID/mixer; khi trôi vị trí ngang thì kiểm tra position PID và quy ước dấu roll/pitch.

Chapter 2: Sử dụng và chạy mô phỏng trong Isaac Sim

2.1 Tạo cây thư mục giống project

Project F450 được tổ chức theo kiểu một ROS 2 workspace, bên trong có một package mô tả robot tên là `f450_description`. Phần mô hình robot, controller, script chạy trong Isaac Sim và script vẽ đồ thị được đặt chung trong package này.

2.1.1 Cấu trúc thư mục chính

Cấu trúc rút gọn của project:

```
f450/
|-- f450.usd
|-- f450_ver2.usd
|-- f450_ver3.usd
|-- f450_ver3_1.usd
|-- configuration/
|   |-- f450_ver3_base.usd
|   |-- f450_ver3_physics.usd
|   |-- f450_ver3_robot.usd
|   '-- f450_ver3_sensor.usd
|-- docs/
|   |-- Baocao.tex
|   |-- Baocao.pdf
|   |-- figures/
|   '-- sections/
'-- ros2_ws/
    |-- build/
    |-- install/
    |-- log/
    '-- src/
        '-- f450_description/
            |-- CMakeLists.txt
            |-- package.xml
            |-- urdf/
            |   '-- f450.urdf.xacro
            |-- meshes/
            |   |-- bottom_plate.stl
            |   |-- up_plate.stl
            |   |-- leg.stl
            |   |-- leg_mount.stl
            |   '-- protectors.stl
            '-- src/
                |-- f450_controller/
                |-- scripr_editor/
                |-- data/
                '-- test/
```

Trong đó:

- `urdf/`: chứa file mô tả robot.
- `meshes/`: chứa các file STL được URDF tham chiếu.

- `src/f450_controller/`: chứa toàn bộ controller Python.
- `src/scripr_editor/`: chứa các đoạn script dùng trong Isaac Sim Script Editor.
- `src/data/`: chứa CSV log và các script vẽ đồ thị.
- `configuration/`: chứa các USD cấu hình được tách theo robot, physics và sensor.

2.1.2 Tạo workspace và package

Nếu tạo lại từ đầu, có thể bắt đầu bằng cây thư mục như sau:

```
mkdir -p f450/ros2_ws/src
cd f450/ros2_ws/src

ros2 pkg create f450_description --build-type ament_cmake

cd f450_description
mkdir -p urdf meshes launch
mkdir -p src/f450_controller
mkdir -p src/scripr_editor
mkdir -p src/data
mkdir -p src/test
```

Sau đó đặt các file vào đúng vị trí:

```
# Robot description
cp f450.urdf.xacro urdf/

# Mesh files used by the URDF
cp *.stl meshes/

# Python controller modules
cp *.py src/f450_controller/

# Isaac Sim Script Editor snippets
cp run.py set_pos.py circle_trajectory.py circle_run.py src/scripr_editor/

# Plotting and logging scripts
cp plot_tracking.py live_plot_tracking.py plot_trajectory_3d.py src/data/
```

Trong `CMakeLists.txt`, project cần install các thư mục mô tả robot:

```
install(
  DIRECTORY urdf meshes launch
  DESTINATION share/${PROJECT_NAME}
)
```

Sau khi chuẩn bị xong, build workspace:

```
cd f450/ros2_ws
colcon build
source install/setup.bash
```

2.2 Import file URDF vào Isaac Sim

File mô tả robot nằm trong thư mục:

```
f450/ros2_ws/src/f450_description/urdf/f450.urdf.xacro
```

Isaac Sim URDF Importer thường làm việc trực tiếp với file `.urdf`. Nếu file hiện tại là `.urdf.xacro`, cần generate ra `.urdf` trước:

```
cd f450/ros2_ws
source install/setup.bash

ros2 run xacro xacro \
  src/f450_description/urdf/f450.urdf.xacro \
  -o src/f450_description/urdf/f450.urdf
```

Sau đó trong Isaac Sim:

1. Mở Isaac Sim.
2. Bật extension URDF Importer nếu chưa bật. Có thể tìm `omni.importer.urdf` trong cửa sổ Extensions.
3. Mở công cụ URDF Importer.
4. Chọn file `f450.urdf`.
5. Kiểm tra đường dẫn mesh để các file STL trong thư mục `meshes/` được load đúng.
6. Import robot vào stage.
7. Lưu stage thành file USD, ví dụ `f450.usd` hoặc `f450_ver3.usd`.

Sau khi import, cần kiểm tra path của rigid body chính. Trong controller hiện tại, path mặc định là:

```
base_link_path = "/f450_simple/base_link"
```

Nếu trong stage Isaac Sim tên prim khác, cần sửa `base_link_path` khi tạo controller.

2.3 Viết controller ngoài và gọi từ Script Editor

Controller được viết bên ngoài Isaac Sim trong thư mục:

```
f450/ros2_ws/src/f450_description/src/f450_controller
```

Thư mục này là một Python package vì có file:

```
f450_controller/__init__.py
```

Isaac Sim Script Editor không tự biết đường dẫn project, nên khi chạy cần thêm đường dẫn controller vào `sys.path`. File `run.py` trong project đang làm đúng việc này.

2.3.1 Setup path và import controller

Đoạn đầu tiên cần chạy trong Script Editor:

```
import sys
import importlib
import os

CONTROLLER_PATH = "/config/Desktop/IssacSim_TA/f450/ros2_ws/src/f450_description/
src"
SCRIPT_PATH = CONTROLLER_PATH + "/scripr_editor"
DATA_DIR = CONTROLLER_PATH + "/data"
TRACKING_CSV = DATA_DIR + "/f450_tracking.csv"

for path in (CONTROLLER_PATH, SCRIPT_PATH):
    if path not in sys.path:
        sys.path.insert(0, path)

os.makedirs(DATA_DIR, exist_ok=True)
importlib.invalidate_caches()
```

Nếu project nằm ở đường dẫn khác, chỉ cần sửa biến CONTROLLER_PATH.

2.3.2 Reload module khi sửa code

Khi đang mở Isaac Sim, nếu sửa file controller bên ngoài, nên reload lại module trước khi tạo controller mới:

```
try:
    f450_app.stop()
except Exception:
    pass

MODULE_NAMES = [
    "f450_controller.control_utils",
    "f450_controller.motor_model",
    "f450_controller.altitude_hold",
    "f450_controller.attitude_pid",
    "f450_controller.position_hold",
    "f450_controller.yaw_hold",
    "f450_controller.motor_mixer",
    "f450_controller.propeller_spinner",
    "f450_controller.tracking_logger",
    "f450_controller.attitude_hold_compat",
    "f450_controller.issac_attitude_hold",
]

modules = {}
for module_name in MODULE_NAMES:
    modules[module_name] = importlib.import_module(module_name)

for module_name in MODULE_NAMES:
    modules[module_name] = importlib.reload(modules[module_name])

issac_attitude_hold = modules["f450_controller.issac_attitude_hold"]
```


Việc reload giúp Script Editor dùng phiên bản code mới nhất mà không cần tắt mở lại Isaac Sim.

2.3.3 Tạo controller và bắt đầu chạy

Sau khi import, tạo đối tượng controller:

```
f450_app = issac_attitude_hold.F450AttitudeHold(
    base_link_path="/f450_simple/base_link",
    z_target=1.0,
    pwm_hover=1650.0,
)

f450_app.position_angle_limit_deg = 6.5
f450_app.position_accel_limit = 1.2

f450_app.set_yaw_target(0.0)
f450_app.start_tracking_log(TRACKING_CSV, sample_period=0.02)
f450_app.start()

print("F450 controller STARTED")
print("Tracking CSV:", TRACKING_CSV)
```

Khi gọi `f450_app.start()`, controller đăng ký callback `on_physics_step(dt)` với physics engine của Isaac Sim. Từ thời điểm đó, mỗi physics step sẽ đọc trạng thái drone, tính PID, mix PWM, cập nhật motor model và apply lực vào body.

2.4 Mở, chạy mô phỏng, log data và vẽ đồ thị

2.4.1 Mở mô phỏng

Quy trình chạy mô phỏng:

1. Mở Isaac Sim.
2. Mở stage USD đã import robot, ví dụ `f450.usd` hoặc `f450_ver3.usd`.
3. Mở Script Editor.
4. Paste hoặc mở nội dung file `src/scripr_editor/run.py`.
5. Kiểm tra lại `CONTROLLER_PATH` và `base_link_path`.
6. Nhấn Run trong Script Editor.

Nếu mọi thứ đúng, terminal/console sẽ in:

```
F450 controller STARTED
Tracking CSV: /config/Desktop/IssacSim_TA/f450/ros2_ws/src/f450_description/src/
data/f450_tracking.csv
```

2.4.2 Đặt setpoint tĩnh

Để đổi target vị trí ngang, chạy trong Script Editor:

```
f450_app.set_position_target(5.0, -1.0)
```

Lệnh này đặt:

$$x_d = 5.0, \quad y_d = -1.0 \quad (288)$$

Để đổi đồng thời x, y, z :

```
f450_app.set_xyz_target(  
    x_target=2.0,  
    y_target=1.0,  
    z_target=1.5,  
)
```

Để chỉ đổi độ cao:

```
f450_app.set_target(1.5)
```

Để đổi yaw target:

```
f450_app.set_yaw_target(0.0)
```

2.4.3 Bay theo quỹ đạo tròn

Project có file:

```
src/scripr_editor/circle_trajectory.py  
src/scripr_editor/circle_run.py
```

Sau khi đã chạy run.py để tạo f450_app, có thể chạy:

```
import circle_trajectory as ct  
import importlib  
importlib.reload(ct)  
  
ct.CIRCLE_RADIUS = 2.0  
ct.CIRCLE_Z = 1.5  
ct.CIRCLE_PERIOD = 20.0  
ct.CIRCLE_CENTER_X = 0.0  
ct.CIRCLE_CENTER_Y = 0.0  
ct.TAKEOFF_TIME = 5.0  
ct.LOOKAHEAD_DT = 0.18  
  
ct.TRACKING_CSV = TRACKING_CSV  
ct.start(f450_app)
```

Khi muốn dừng quỹ đạo tròn và lưu log:

```
import circle_trajectory as ct  
ct.stop(f450_app)
```

2.4.4 Bật và dừng log

Để bật log tracking:

```
DATA_DIR = CONTROLLER_PATH + "/data"
TRACKING_CSV = DATA_DIR + "/f450_tracking.csv"
f450_app.start_tracking_log(TRACKING_CSV, sample_period=0.02)
```

Với `sample_period=0.02`, logger ghi dữ liệu tối đa khoảng:

$$f_s = \frac{1}{0.02} = 50 \text{ Hz} \quad (289)$$

Để dừng log:

```
f450_app.stop_tracking_log()
```

File CSV sẽ chứa các nhóm cột chính:

- Position: `x`, `x_target`, `y`, `y_target`, `z`, `z_target`.
- Attitude degree: `roll_deg`, `roll_target_deg`, `pitch_deg`, `pitch_target_deg`, `yaw_deg`, `yaw_target_deg`.
- Attitude radian: `roll_rad`, `roll_target_rad`, `pitch_rad`, `pitch_target_rad`, `yaw_rad`, `yaw_target_rad`.

2.4.5 Vẽ đồ thị realtime

Trong lúc Isaac Sim đang chạy, mở terminal khác và chạy:

```
python3 ~/Desktop/IssacSim_TA/f450/ros2_ws/src/f450_description/src/data/
live_plot_tracking.py \
~/Desktop/IssacSim_TA/f450/ros2_ws/src/f450_description/src/data/f450_tracking.
csv \
--wait \
--max-points 2000
```

Option `-wait` cho phép script đợi tới khi file CSV xuất hiện. Option `-max-points` giới hạn số điểm hiển thị để đồ thị realtime không quá nặng.

2.4.6 Vẽ đồ thị 2D sau mô phỏng

Sau khi dừng mô phỏng và gọi `stop_tracking_log()`, tạo đồ thị 2D:

```
python3 ~/Desktop/IssacSim_TA/f450/ros2_ws/src/f450_description/src/data/
plot_tracking.py \
~/Desktop/IssacSim_TA/f450/ros2_ws/src/f450_description/src/data/f450_tracking.
csv
```

Đồ thị 2D dùng để so sánh target và giá trị thực của:

$$x, y, z, roll, pitch, yaw \quad (290)$$

Vị trí dùng đơn vị mét, còn góc được vẽ theo độ.

2.4.7 Vẽ quỹ đạo 3D

Để xem quỹ đạo bay trong không gian 3D sau mô phỏng:

```
python3 ~/Desktop/IssacSim_TA/f450/ros2_ws/src/f450_description/src/data/  
plot_trajectory_3d.py \  
~/Desktop/IssacSim_TA/f450/ros2_ws/src/f450_description/src/data/f450_tracking.  
csv \  
--show
```

Để xem live 3D trong khi Isaac Sim đang chạy:

```
python3 ~/Desktop/IssacSim_TA/f450/ros2_ws/src/f450_description/src/data/  
plot_trajectory_3d.py \  
~/Desktop/IssacSim_TA/f450/ros2_ws/src/f450_description/src/data/f450_tracking.  
csv \  
--live --wait --interval 500
```

Để vẽ thêm vector attitude dọc theo đường bay:

```
python3 plot_trajectory_3d.py f450_tracking.csv --show --attitude --attitude-step  
15
```

Mặc định output ảnh sẽ có dạng:

```
f450_tracking_tracking.png  
f450_tracking_traj3d.png
```

2.4.8 Dừng controller

Khi kết thúc thử nghiệm, nên dừng log và controller:

```
f450_app.stop_tracking_log()  
f450_app.stop()
```

Nếu muốn chạy lại từ đầu, chạy lại file `run.py`. Trong file này đã có đoạn:

```
try:  
    f450_app.stop()  
except Exception:  
    pass
```

Nhờ đó controller cũ được dừng trước khi tạo controller mới, tránh trường hợp nhiều callback physics chạy chồng lên nhau.

References

- [1] K. J. Astrom and T. Hagglund, *PID Controllers: Theory, Design, and Tuning*. Instrument Society of America, 1995.
- [2] S. Bouabdallah, “Design and control of quadrotors with application to autonomous flying,” Ph.D. dissertation, Ecole Polytechnique Federale de Lausanne, 2007.
- [3] R. Mahony, V. Kumar, and P. Corke, “Multirotor aerial vehicles: Modeling, estimation, and control of quadrotor,” *IEEE Robotics & Automation Magazine*, vol. 19, no. 3, pp. 20–32, 2012.